A
Project Report On

Cognitive Pattern Decoder

By

Manan Javiya (CE099)(23CEUOZ060)
Maitrey Patel(CE126)(23CEUOZ113)
of
B.Tech CE , Semester VI

Subject: System Design Practice

Guided by:
Dr. Hariom A. Pandya
Associate Professor
CE Department

Faculty of Technology
Department of Computer Engineering
Dharmsinh Desai University
College Road
Nadiad – 387001

CERTIFICATE

This is to certify that the practical/term work carried out in the subject of System Design Practice and recorded in this report is the bonafide work of

Manan Javiya (CE099)(23CEUOZ060)
Maitrey Patel(CE126)(23CEUOZ113)

Of B.Tech semester VI in the branch of Computer Engineering during the academic year 2025-2026.

Dr. Hariom Pandya,
Associate Professor,
CE Department,
Dharmsinh Desai
University, Nadiad.

Dr. C.K. Bhensdadia,
Head,
CE Department,
Dharmsinh Desai
University, Nadiad.

Comprehensive Project Report and SRS

Cognitve Pattern Decoder

A system that analyzes data patterns to idenify and interpret meaningful insights.

Academic Year: 2025–2026

Department: Computer Engineering

Institution: Dharmsinh Desai University

Project Team: Javiya Manan, Patel Maitrey

Guide: Hariom A. Pandya

1 Abstract

Cognitive Pattern Decoder is an intelligent system designed to capture and analyze a programmer's coding behaviour using fine-grained interaction metrics such as typing speed, backspace usage, paste operations, file switches, cursor movements, and pauses. These metrics are processed by a machine learning (ML) model to infer cognitive coding patterns and present them through intuitive dashboards and summaries.

The platform consists of a React-based frontend dashboard, an Express.js + MongoDB backend API, and a dedicated Python-based ML microservice. A Visual Studio Code extension is used to collect real-time coding activity data and send it securely to the backend.

This report presents the problem context, objectives, system architecture, data model, functional and non-functional requirements, API contracts, implementation details, testing approach, deployment model, and potential future enhancements of the Cognitive Pattern Decoder project.

---

2 Introduction

2.1 Background

Programming behaviour during active development sessions reveals a lot about a developer's cognitive style: how they plan, debug, refactor, and manage their focus. Traditional productivity tools measure only coarse metrics such as total time or lines of code written, which do not capture the deeper cognitive patterns underlying coding behaviour.

With modern editors and extensions, it is possible to log richer signals like keystrokes, backspace patterns, editing pauses, file switches, and save operations. Combined with machine learning, these signals can be transformed into meaningful insights such as:

- Cognitive load and fatigue tendencies
- Preferred working style (systematic vs. intuitive)
- Balance between coding, debugging, and planning

However, there is a lack of end-to-end systems that capture this data, process it with ML, and present it in a form that is understandable and actionable for developers.

2.2 Problem Statement

Current tools for monitoring coding behaviour have several limitations:

- Limited detail: Existing analytics often count only commits or simple time-ontask.
- No cognitive interpretation: Metrics are not translated into understandable cognitive styles or patterns.
- No integrated pipeline: Data collection, storage, ML analysis, and visualization are not integrated into one coherent system.
- Lack of personalization: Little support for per-user history, trends, and personalized insights.

There is a need for a system that can automatically capture detailed coding metrics, infer cognitive patterns using ML, and present clear, visual feedback to the programmer.

2.3 Project Goal

The goal of Cognitive Pattern Decoder is to build an end-to-end platform that:

- Captures detailed coding metrics from real development sessions.
- Sends these metrics to a backend that stores and organizes them by user and project.
- Calls a Python ML service to infer cognitive clusters and traits.
- Produces dashboards, timelines, and textual summaries describing a developer's cognitive coding patterns and areas for improvement.

---

3 Objectives

- O1: Implement secure user registration, authentication, and profile management.
- O2: Capture session-level coding metrics (typing speed, backspaces, paste count, etc.) from a VS Code extension or frontend.
- O3: Persist sessions, projects, ML results, and notifications in MongoDB using a normalized schema.
- O4: Integrate a Python-based ML service that receives metrics and returns a cognitive cluster label.
- O5: Generate dashboards and summarized analytics (focus, consistency, memory, logic, etc.) per user.
- O6: Provide history and comparison views showing how a user's pattern evolves over time.

- O7: Trigger personalized notifications/alerts for events such as fatigue or distraction patterns.

---

4 Scope and Constraints

4.1 Scope

- In Scope
- Web backend API for user auth, session creation, ML result storage, history, and notifications.
- React frontend dashboard for viewing real-time metrics, past sessions, and cognitive analysis.
- Integration with a Python ML microservice for clustering sessions based on metrics.
- Basic role separation: authenticated end users, with optional admin views through analysis endpoints.
- Visual charts (e.g., bar/line charts) for trend and distribution visualization.

4.2 Current Constraints

- The ML pipeline uses a separate service at http://localhost:8000/predict and assumes it is available.
- The system is optimized for single-institution / single-tenant use (no multi-tenant governance yet).
- Performance is tuned for individual developers or small groups, not yet for very large organizations.
- Clinical/psychological validation of the cognitive labels is out of scope for this academic project.

---

5 System Overview

5.1 High-Level Context

Key components:

- Frontend Web App (React + Vite) – user dashboard and visualizations.

- Backend API (Express + MongoDB) – all REST endpoints, authentication, and data storage.
- Python ML Service – separate microservice that receives metrics and returns a cluster label.
- VS Code Extension – optional component to stream metrics from the editor to the backend.

<<FIGURE 1: System Context Diagram – Frontend, Backend API, MongoDB, Python ML Service, VS Code Extension>>

5.2 User Roles

- Developer (End User):
- Registers and logs into the platform.
- Runs coding sessions that are tracked.
- Views dashboards, cognitive summaries, and history.
- Admin/Analyst (implicit role via analysis routes):
- Can query user performance data via email.
- Reviews trends and summary statistics.

5.3 Use Cases

Main use cases:

- Developer logs in and starts a coding session.
- Metrics are recorded during the session and submitted to backend.
- Developer ends the session; backend calls ML to compute cognitive cluster.
- Developer views dashboard metrics, cluster meaning, and narrative summary.

- Developer checks cognitive history across multiple sessions.
- Developer receives notifications such as fatigue or distraction alerts.

<<FIGURE 2: Use Case Diagram – Developer and Admin use cases>>

---

6 Architecture and Design

6.1 Technology Stack

Backend

- Node.js
- Express.js
- MongoDB + Mongoose
- JSON Web Tokens (jsonwebtoken)
- Password hashing (bcryptjs)
- File uploads (multer)
- HTTP client (axios)
- CORS, express-rate-limit Frontend

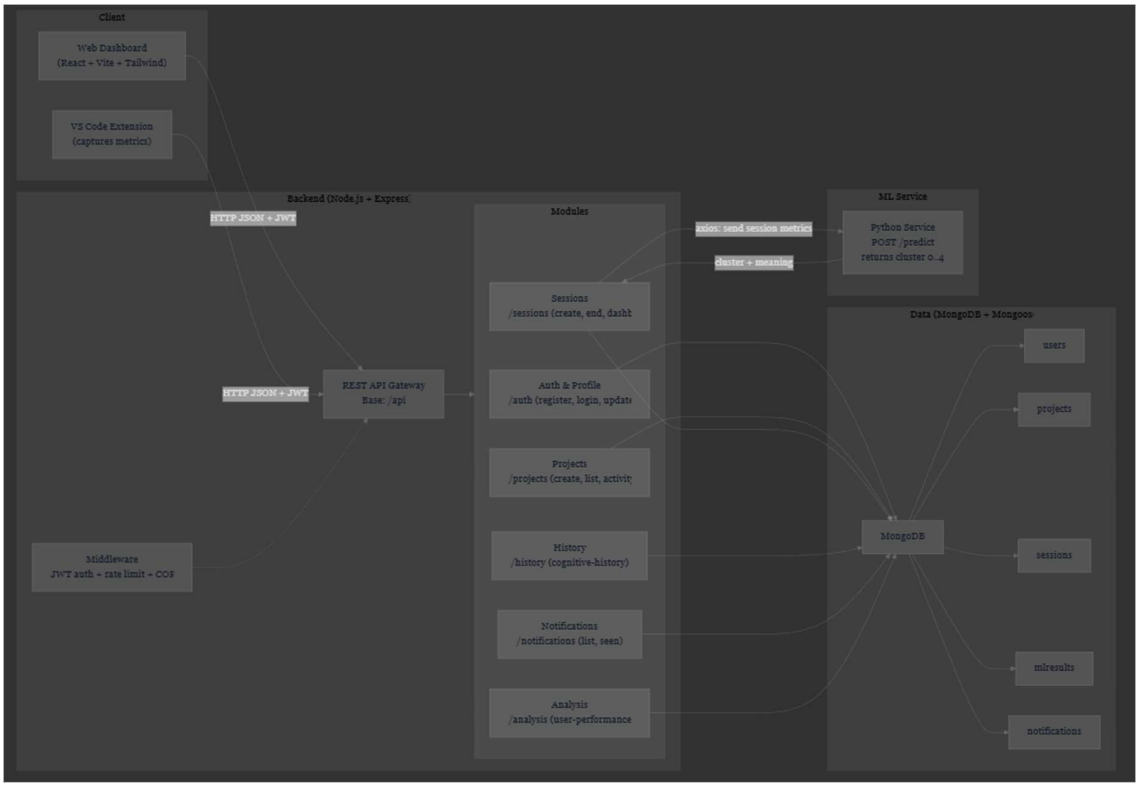
- React 18
- Vite build tool
- Tailwind CSS
- react-router-dom
- chart.js + react-chartjs-2
- lucide-react, react-icons

ML / Analysis

- Python-based ML service (cluster analysis) exposed via /predict endpoint.
- Developer Tooling
- VS Code extension in vs-code-extension for metric collection.

6.2 System Components

- Auth and User Management
- Handles registration, login, JWT issue, and profile updates (with profile image upload).
- Projects and Context
- Each user can have one or more projects, used to contextualize sessions.
- Sessions and Metrics Capture
- Each coding session records detailed behaviour metrics and duration.
- ML Result and Cognitive Pattern Storage
- Results from the ML service are stored per session and user, including cluster meaning.
- Notifications Service
- Generates and stores notifications based on ML results and heuristics.
- Analytics and Dashboard
- Server-side computations provide high-level metrics and narrative summaries. <<FIGURE 3: Component Diagram – Auth, Projects, Sessions, ML, Notifications, Dashboard>>



6.3 Core Data Model

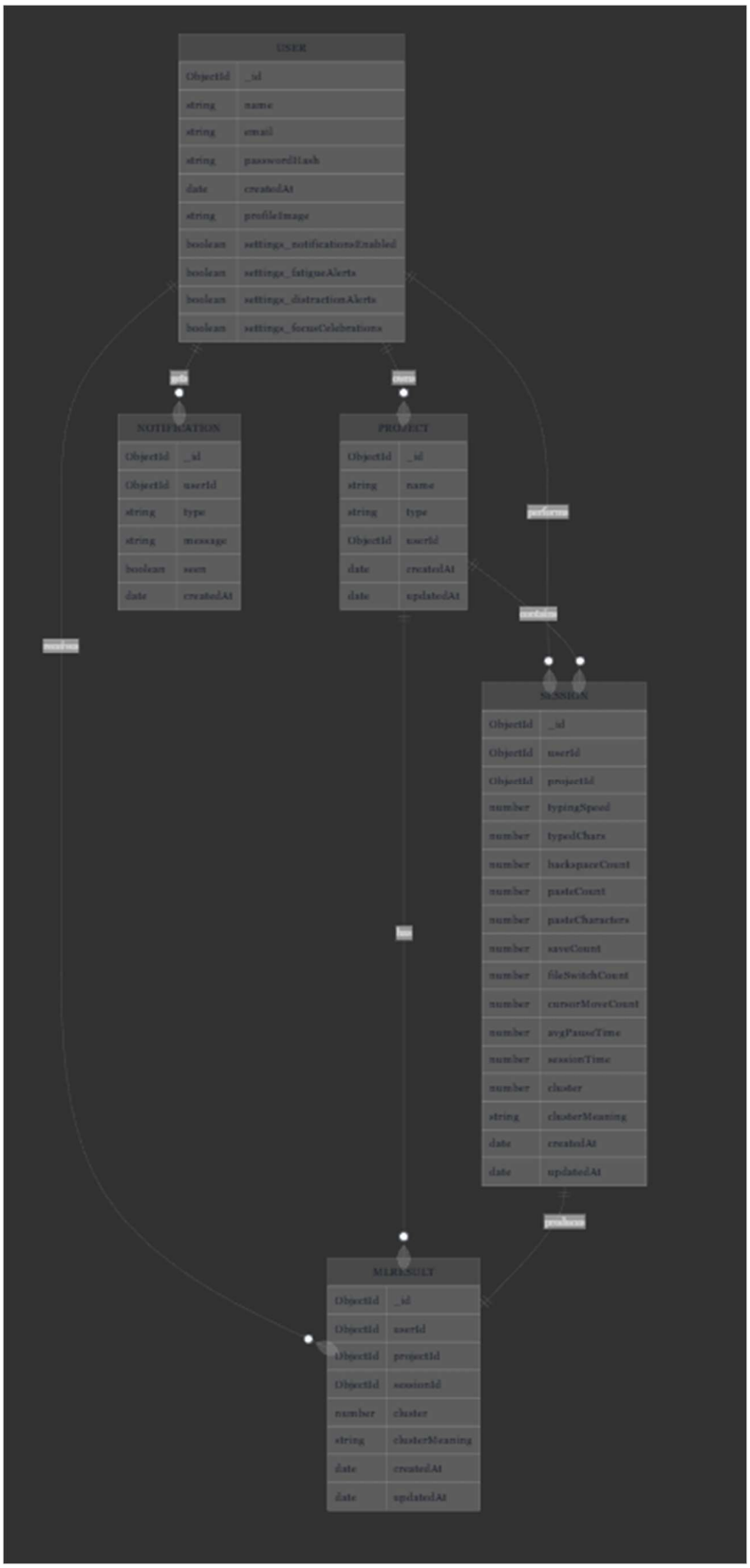
Collections (MongoDB):

- User
- name: String
- email: String (unique, required)
- passwordHash: String
- createdAt: Date
- profileImage: String
- settings
- notificationsEnabled: Boolean
- fatigueAlerts: Boolean
- distractionAlerts: Boolean
- focusCelebrations: Boolean
- name: String
- type: String (enum: DSA, WEB, ML, DEBUG, mobile, dsa)
- userId: ObjectId → User

- createdAt, updatedAt: Date
- Session
- userId: ObjectId → User
- projectId: ObjectId → Project
- typingSpeed: Number
- typedChars: Number
- backspaceCount: Number
- pasteCount: Number
- pasteCharacters: Number
- saveCount: Number
- fileSwitchCount: Number
- cursorMoveCount: Number
- avgPauseTime: Number
- sessionTime: Number
- cluster: Number | null
- clusterMeaning: String | null
- createdAt, updatedAt: Date

- MLResult
- userId: ObjectId → User
- projectId: ObjectId → Project
- sessionId: ObjectId → Session
- cluster: Number
- clusterMeaning: String
- createdAt, updatedAt: Date
- Notification
- userId: ObjectId → User
- type: String (focus, fatigue, distraction, improvement)
- message: String
- seen: Boolean
- createdAt: Date

<<FIGURE 4: ER Diagram – User, Project, Session, MLResult, Notification>>

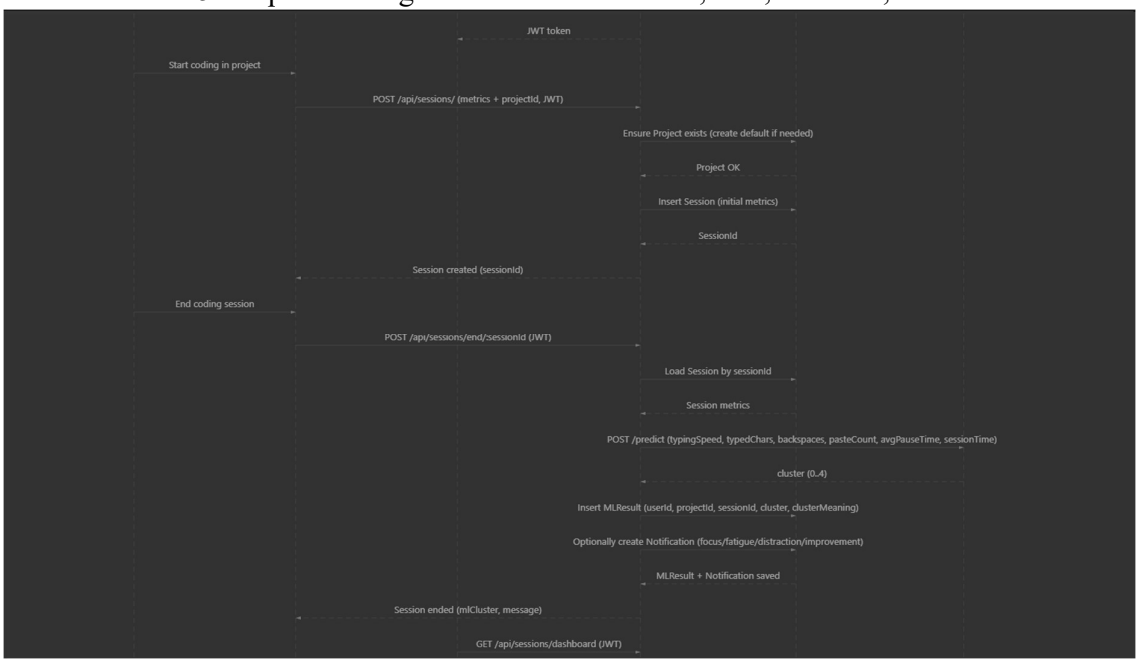


6.4 Cognitive Session Lifecycle

High-level lifecycle:

1. User starts a coding session.
2. Session creation endpoint stores initial metrics and context (project, user).
3. During/after session, metrics are finalized and session is ended.
4. Backend calls ML service with metrics to get cluster.
5. ML result and interpreted meaning are stored in MLResult.
6. Dashboard summary combines session and MLResult to compute analytics.
7. Notifications may be generated based on cluster or thresholds.

<<FIGURE 5: Sequence Diagram – Session Creation, End, ML Call, and Dashboard>>



---

7 Implementation Details

7.1 Authentication and Authorization

- JWT-based login via /api/auth/login.
- Passwords stored as bcrypt hashes.

- Protected routes use an authMiddleware that validates tokens and attaches req.user.
- Profile update supports uploading a profile image via multer.

7.2 Session Capture and Metrics Collection

- Detailed metrics (typingSpeed, typedChars, backspaceCount, pasteCount, cursorMoveCount, etc.) are sent as JSON to POST /api/sessions/.
- Backend validates metrics and confirms that a valid project exists, creating a default project if necessary.
- A Session document is created and associated with the user.

7.3 Cognitive Pattern Analysis Pipeline

- POST /api/sessions/end/:sessionId is called when the user ends the session.
- Backend loads the session metrics and calls the Python ML service at /predict.
- The ML service returns a cluster id (0–4).
- The backend maps the cluster to a human-readable meaning such as “Systematic Thinker”, “Creative Coder”, etc.
- An MLResult is created and stored, linking user, project, and session.

7.4 Dashboard and Analytics Computation

- GET /api/sessions/dashboard computes:
- Derived metrics: WPM, accuracy, error rate, paste ratio, keypress latency.
- Behaviour distributions: coding vs debugging vs planning.
- Cognitive scores: focusScore, consistencyScore, memoryScore, logicScore.
- Narrative summaries and comparison narratives based on history.
- This endpoint powers the React dashboard graphs and summary cards.

7.5 VS Code Extension Integration

- A VS Code extension (in vs-code-extension) is used to track coding activity and communicate with the backend.
- It can send metrics to session endpoints and help automate session start/end flows.

---

8 Functional Requirements Summary

High-level summary (to be formatted as a table in final document):

- Authentication – User registration, login, profile updates.
- Projects – Create and list user projects, fetch project activity.

  Sessions – Create and end sessions with metrics and ML analysis.
- ML Results – Store and query ML-based cognitive patterns.
- Notifications – Generate and manage user notifications.
- History & Analysis – Provide cognitive history and user performance analytics.
- Dashboard – Display real-time and historical cognitive metrics.

---

9 Detailed Software Requirements

9.1 Definitions and Acronyms

- CPD: Cognitive Pattern Decoder
- ML: Machine Learning
- JWT: JSON Web Token
- API: Application Programming Interface UI: User Interface

9.2 Functional Requirements

9.2.1 Authentication and Profile

- FR-AUTH-01: The system shall allow registration of new users via POST /api/auth/register.
- FR-AUTH-02: The system shall allow users to log in via POST /api/auth/login and receive a JWT.
- FR-AUTH-03: The system shall allow authenticated users to update their profile and profile image via PUT /api/auth/update.
- FR-AUTH-04: The system shall protect all session, project, notification, and analysis routes using authentication middleware.

9.2.2 Projects and Coding Context

- FR-PROJ-01: The system shall create a project for a user via POST /api/projects/.

-
- FR-PROJ-02: The system shall list all projects for a user via GET /api/projects/user/:userId.
- FR-PROJ-03: The system shall provide project activity details via GET /api/projects/:projectId/activity.

9.2.3 Sessions and Cognitive Metrics

- FR-SES-01: The system shall create a session via POST /api/sessions/ given valid metrics and user context.

  FR-SES-02: The system shall validate that metric values are non-negative and sessionTime > 0.

- FR-SES-03: The system shall ensure a valid project is associated to each session, creating a default if needed.
- FR-SES-04: The system shall allow ending a session via POST /api/sessions/end/:sessionId which triggers ML analysis.
- FR-SES-05: The system shall allow listing sessions by user and project via GET /api/sessions/user/:userId and GET /api/sessions/project/:projectId.
- FR-SES-06: The system shall compute a dashboard summary for the logged-in user via GET /api/sessions/dashboard.

9.2.4 ML Results and Cognitive Patterns

- FR-ML-01: The system shall call the external ML service with session metrics when ending a session.
- FR-ML-02: The system shall create an MLResult containing userId, projectId, sessionId, cluster, and clusterMeaning.
- FR-ML-03: The system shall support optional manual ML result creation via POST /api/mlresults/.
- FR-ML-04: The system shall allow querying ML results by user via GET /api/mlresults/user/:userId.

9.2.5 Notifications and Alerts

- FR-NOTIF-01: The system shall create notifications when particular cognitive patterns or thresholds are detected.
- FR-NOTIF-02: The system shall provide a list of notifications for the current user via GET /api/notifications.
- FR-NOTIF-03: The system shall allow marking notifications as seen via PUT /api/notifications/:id/seen.

9.2.6 Historical Analysis and Comparison

- FR-HIST-01: The system shall provide cognitive history for the current user via GET /api/history/cognitive-history.
- FR-HIST-02: The system shall return for each historical entry: date, project, duration, WPM, backspaces, cluster, and cluster meaning.

9.2.7 User Performance Analysis

- FR-ANL-01: The system shall provide user performance analysis for a specific user identified by email via GET /api/analysis/user-performance/:email.

  FR-ANL-02: The system shall support comparison between a user and aggregate patterns for analysis purposes.

---

10 API Endpoint Catalog

Authentication and Profile

- POST /api/auth/register – Public – Register a new user.
- POST /api/auth/login – Public (rate-limited) – Login and get JWT.
- PUT /api/auth/update – Authenticated – Update user profile and profile image.
- GET /api/auth/compare – Authenticated – Compare the current user's performance with others.

Projects

- POST /api/projects/ – Authenticated – Create project.
- GET /api/projects/user/:userId – Authenticated – List projects for a user.
- GET /api/projects/:projectId/activity – Authenticated – Fetch activity details for a project.

Sessions

- POST /api/sessions/ – Authenticated – Create coding session with metrics.

  POST /api/sessions/end/:sessionId – Authenticated – End session and run ML analysis.

- GET /api/sessions/user/:userId – Authenticated – List sessions by user.
- GET /api/sessions/project/:projectId – Authenticated – List sessions by project.

-
- GET /api/sessions/dashboard – Authenticated – Dashboard summary for loggedin user. History
- GET /api/history/cognitive-history – Authenticated – Cognitive history for loggedin user.
- GET /api/history/user – Authenticated – Alternate history route (same behaviour).

ML Results

- POST /api/mlresults/ – Internal/Admin – Save ML result entry.
- GET /api/mlresults/user/:userId – Authenticated/Admin – Get ML results by user.

Notifications

GET /api/notifications – Authenticated – List notifications for current user.

- PUT /api/notifications/:id/seen – Authenticated – Mark a notification as seen.

Analysis

- GET /api/analysis/user-performance/:email – Authenticated – Get detailed user performance by email.

---

11 External Interface Requirements

11.1 User Interface Requirements

- UI-01: The dashboard must display key metrics (WPM, accuracy, error rate, session time) for the latest session.
- UI-02: The dashboard must visualize trends of typing speed and cognitive scores across sessions.
- UI-03: The history page must list past sessions with date, project, cluster, and key metrics.
- UI-04: The notifications area must list unseen and seen notifications and allow marking as seen.

UI-05: Profile settings must allow toggling notification preferences (fatigue, distraction, etc.) and uploading a profile image.

11.2 Software Interfaces

- Express.js server connecting to MongoDB via Mongoose.
- Python ML service reachable over HTTP (/predict).

-
- VS Code extension calling REST endpoints on the backend.

11.3 Communication Interfaces

- RESTful HTTP/HTTPS APIs with JSON payloads.
- Authentication via Authorization: Bearer <JWT> headers.
- CORS configuration allowing frontend origins such as http://localhost:5173.

---

12 Data Requirements

12.1 Controlled Values

- User.settings flags: true | false.
- Project.type: DSA, WEB, ML, DEBUG, mobile, dsa, defult.
- Notification.type: focus, fatigue, distraction, improvement.
- MLResult.cluster: integers 0–4 mapped to specific meanings.

12.2 Storage and File Rules

- Profile images stored under /uploads directory, served by Express static route.
- All IDs stored as MongoDB ObjectIds.
- Timestamps automatically generated via timestamps: true in Mongoose schemas.

---

13 Non-Functional Requirements

13.1 Security

- Passwords must be stored using bcrypt hashing.

  All protected API routes must validate JWTs.

- Basic rate limiting must be applied to login to prevent brute-force attacks.
- CORS settings must restrict allowed origins in production.

13.2 Performance

- Session creation and ending must complete within a reasonable time (e.g., < 1–2 seconds excluding ML latency).
- Dashboard endpoint must compute metrics using e icient aggregate operations on MongoDB.

13.3 Reliability and Availability

- Backend must handle ML service errors gracefully and return a clear error message instead of crashing.
- If ML is temporarily unavailable, sessions should still be saved and ML results can be computed later.

13.4 Usability

- Dashboards should present metrics and cognitive labels in clear, understandable language.

- History and notifications should be easy to navigate for non-technical users.

---

14 Testing and Validation

14.1 Backend Testing

- Unit tests for:
- Registration and login logic.
- Session input validation.
- ML result creation and error handling.
- Notification creation and listing.

14.2 Frontend Testing

- Manual testing of:
- Navigation across pages.

- Correct display of dashboard metrics.
- History sorting and filtering.
- Notification updates.

14.3 Integration Testing

- End-to-end flow:
- Login → Create Session → End Session → Fetch Dashboard → View History.
- Testing error paths when ML service is down.

14.4 Validation Outcomes

- Confirm that metrics displayed on the frontend match stored backend data.
- Confirm that ML clusters and meanings are synchronized between ML service and backend mapping.

---

15 Deployment and Operations

15.1 Local Development Setup

- Backend:
- Run npm install in backend and start with npm start.
- Environment variables configured in .env file (MongoDB URI, JWT secret, etc.).
- Frontend:
- Run npm install in frontend and start with npm run dev.
- ML Service:
- Run Python environment and start service on http://localhost:8000.

15.2 Production / Containerized Runtime

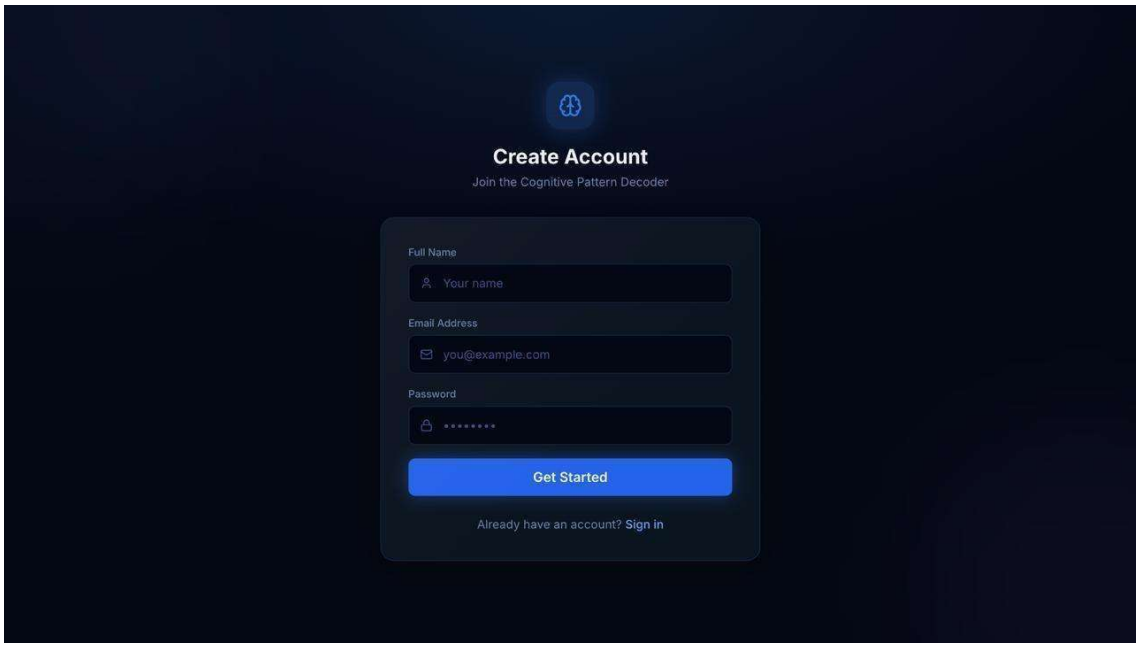
- Use Docker or similar orchestration to run:
- Express backend container.
- React build served via a static server.
- MongoDB service.
- Python ML service container.
- Reverse proxy (e.g., Nginx) to route /api/* to backend and serve frontend assets.

---

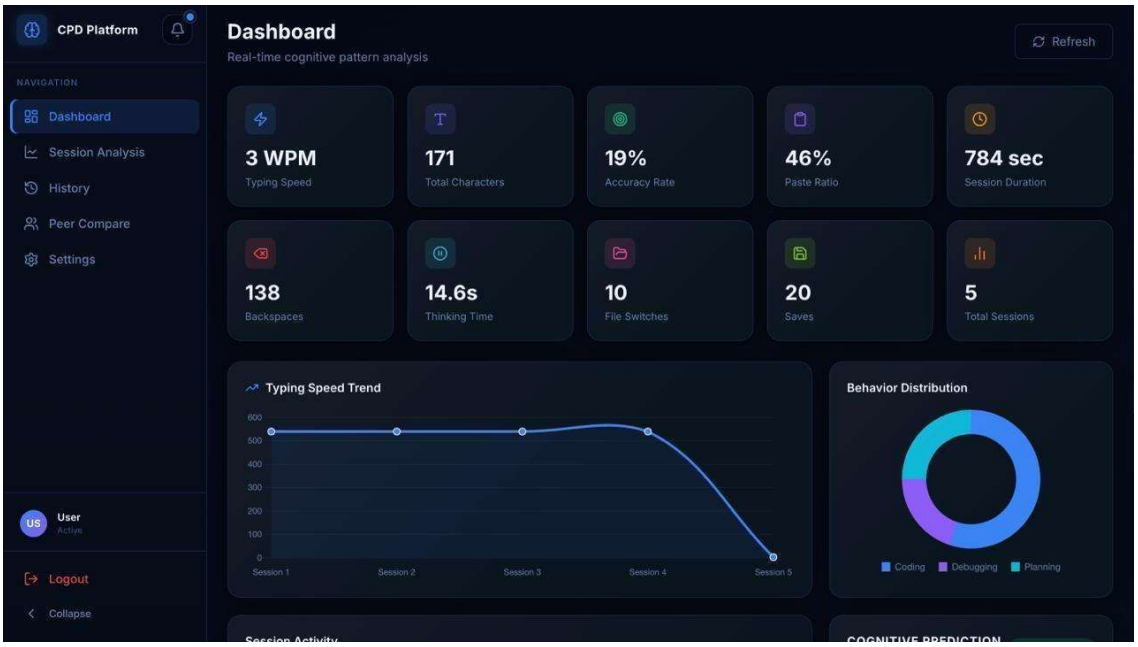
16 Project Screenshots

[PLACEHOLDER LIST – YOU WILL INSERT YOUR REAL SCREENSHOTS]

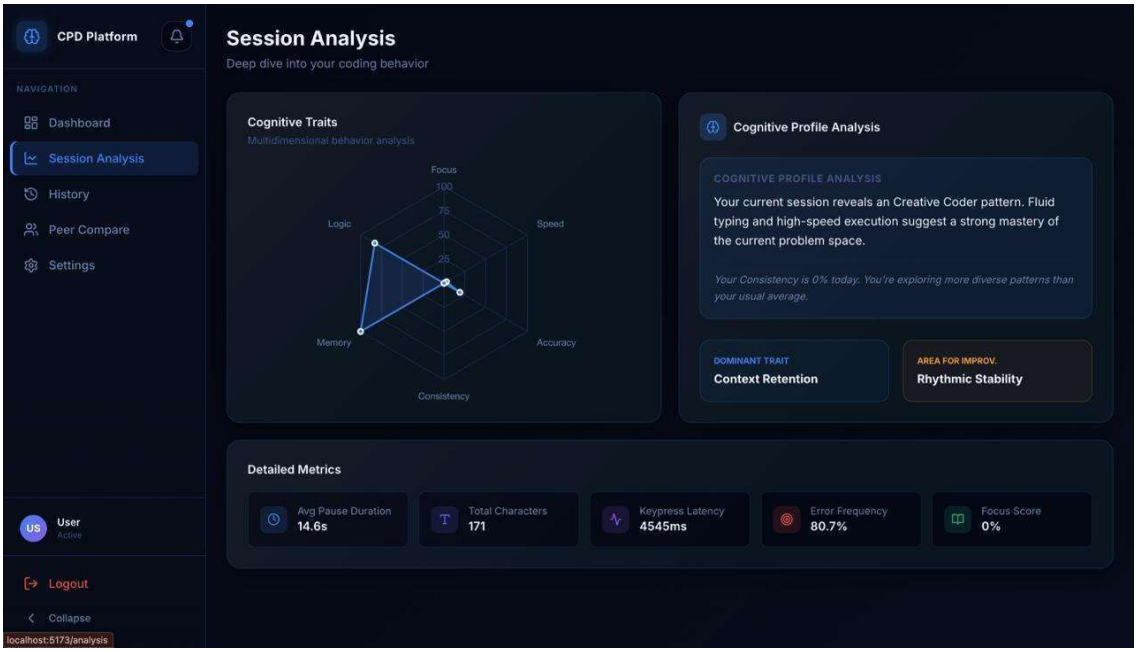
- <<SCREENSHOT 1: Login / Register>>



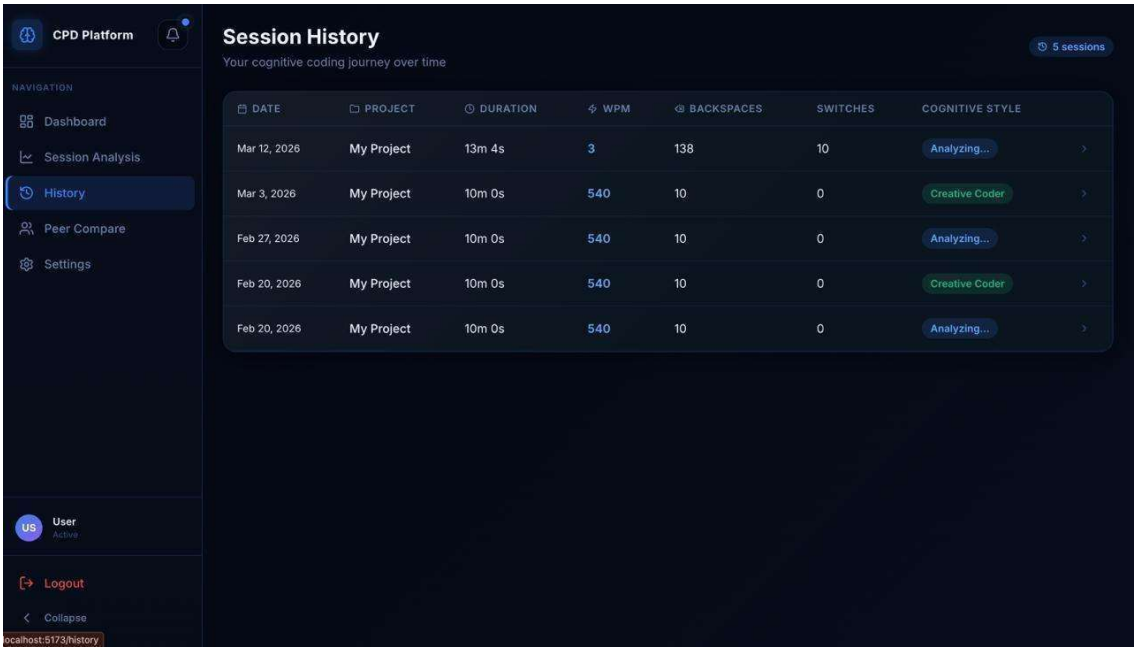
<<SCREENSHOT 2: Dashboard – Summary Cards and Charts>>



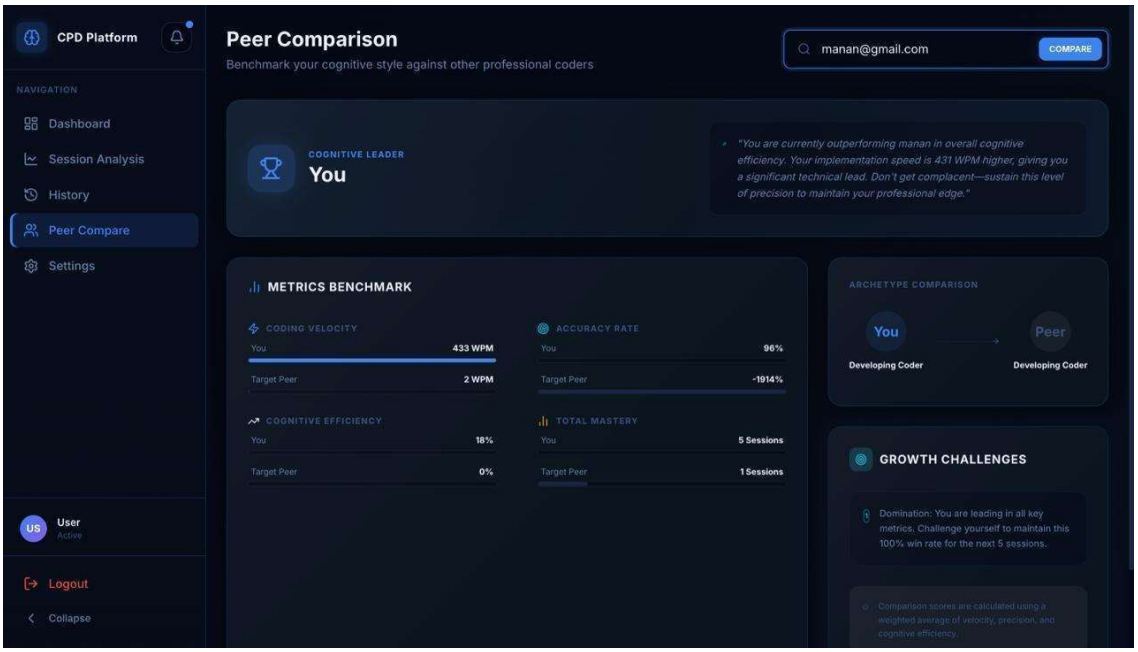
- <<SCREENSHOT 3: Session Analytics>>



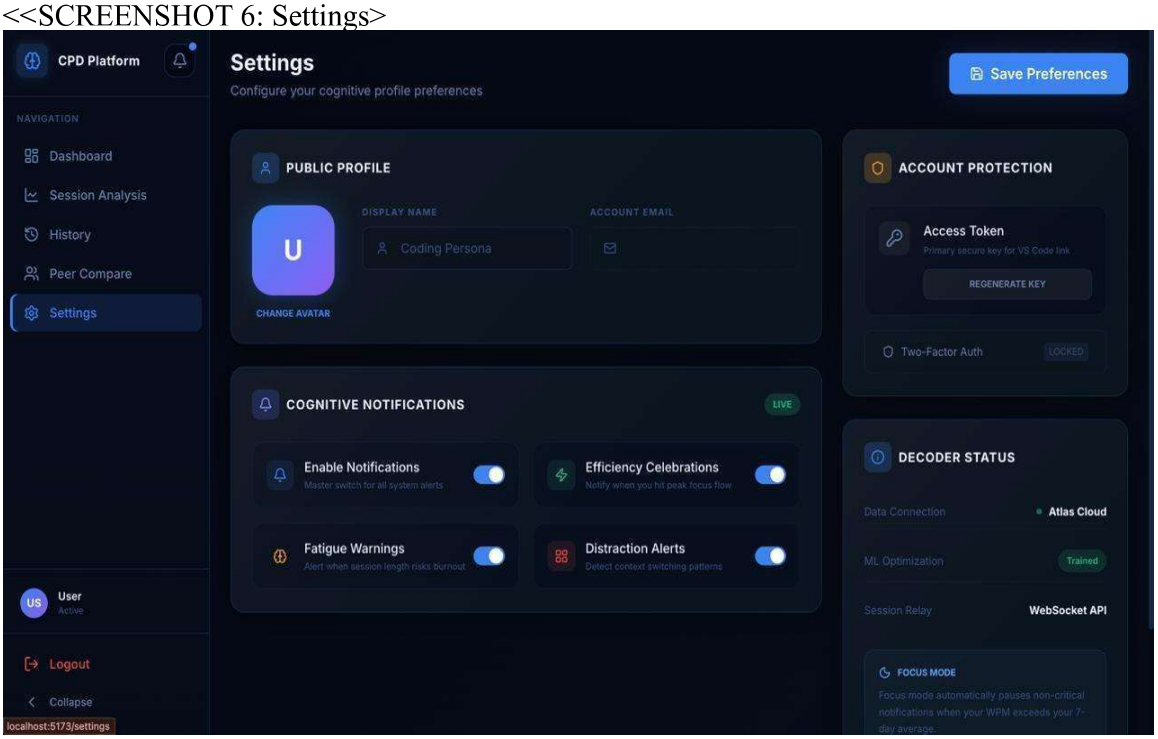
<<SCREENSHOT 4: Session History >>



- <<SCREENSHOT 5: Peer Compare>>



<<SCREENSHOT 6: Settings>



---

17 Conclusion

Cognitive Pattern Decoder presents an integrated solution for capturing, analyzing, and visualizing coding behaviour. By combining detailed low-level metrics, a machine learning clustering model, and a user-friendly dashboard, the system decodes a developer's cognitive style and provides actionable feedback.

The project demonstrates how modern web technologies, MongoDB, and a separate ML service can be orchestrated into a coherent platform suitable for research, personal improvement, or educational settings.

---

18 Future Work

- Enhance ML model sophistication and validate clusters with larger user datasets.
- Add support for more IDEs beyond VS Code.
- Implement advanced comparative analytics across multiple users or teams.
- Introduce goal tracking and personalized improvement recommendations.

- Add automated frontend and backend test suites and continuous integration.

---

19 Appendix A: Environment Baseline

Typical environment variables:

- MONGODB_URI – MongoDB connection string.
- JWT_SECRET – secret key for token signing.
- PORT – backend server port.
- ML_SERVICE_URL – URL for Python ML /predict endpoint.

---

20 Appendix B: Current-State Gaps

- ML model may be trained on limited data and should not be used for clinical assessment.
- Performance and load testing for very high tra ic is not yet performed.
- Multi-tenant and role-based admin dashboards are basic and can be extended.